# Lab 4

$ linked lists: delete + reverse

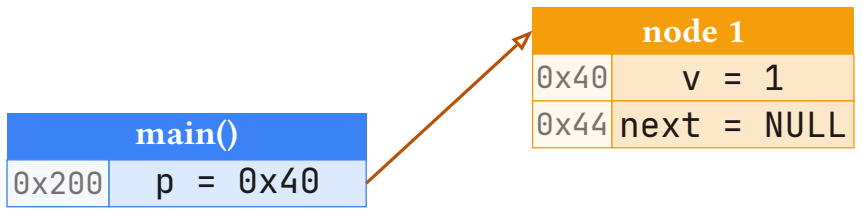
# The two TODOs

```
node *delete_node(node *head, int v);    // Part 1
node *reverse_list(node *head);          // Part 2
```

- Both return the **new** head
- Only edit these two (helpers allowed)

# One node

```c
int main(void) {
  node *p = new_node(1);
  printf("%d\n", p->v);       // 1
  printf("%p\n", p->next);    // (nil)
  free(p);
  return 0;
}
```




Review: memory slides, **Part 7: Linked Lists** github.com/storchdev/cs-materials/.../memory-c-programmer.pdf

```
/* Given code */
```

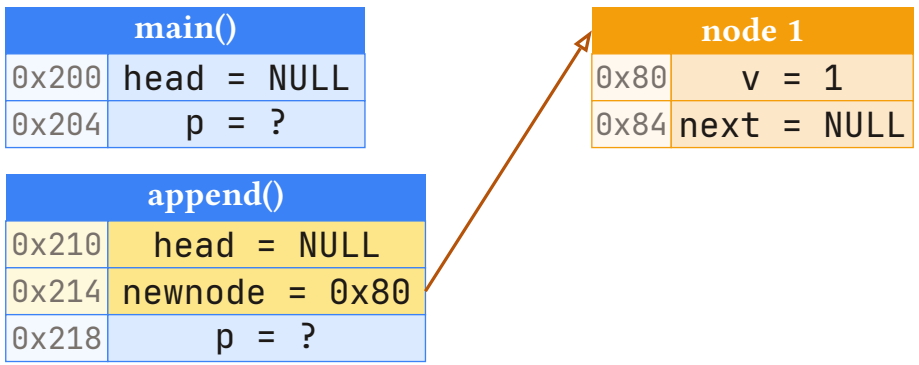
# `main` starts

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```

| main() | |
|---|---|
| 0x200 | head = NULL |
| 0x204 | p = ? |

# append(NULL, 0x80)

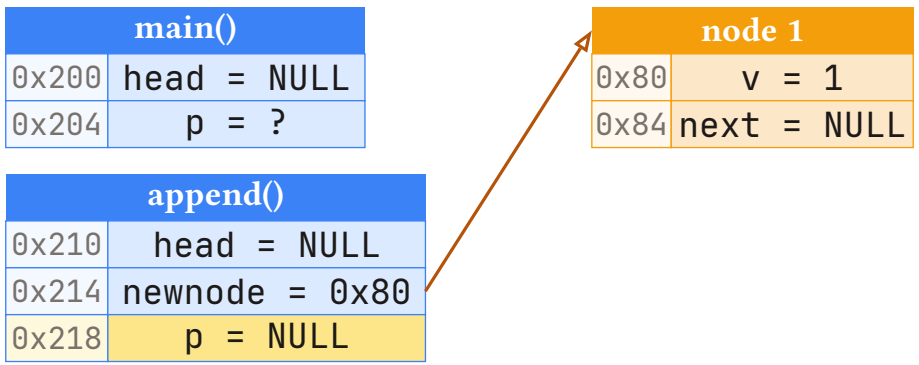
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode->next = NULL;
14    if (p == NULL)
15      return newnode;
16    p->next = newnode;
17    return head;
18  }
```



# p = NULL

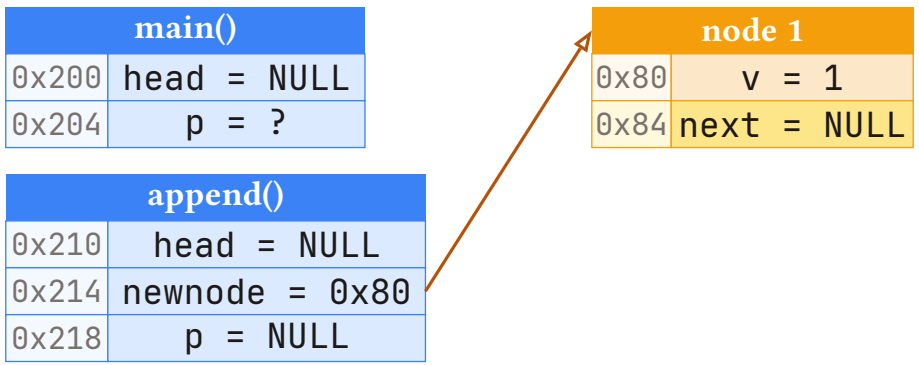
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```

find_last(NULL) ⟶ NULL



# newnode→next = NULL

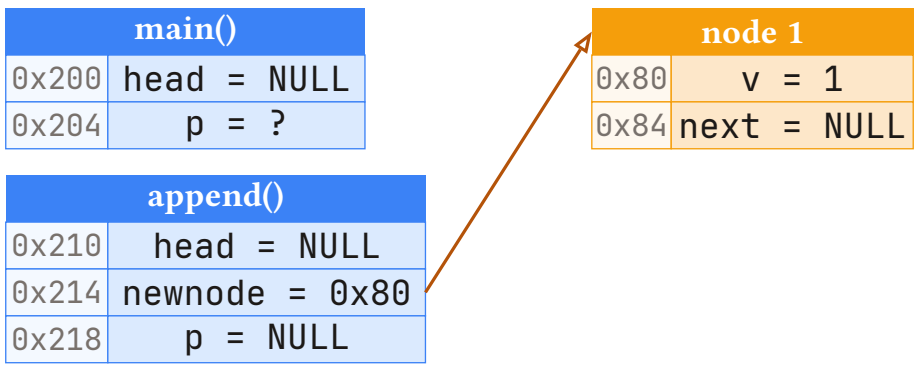
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```




Already `NULL` from `new_node`

# Empty list: `return newnode`

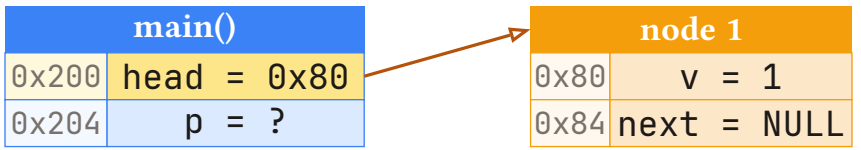
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```



# head = 0x80

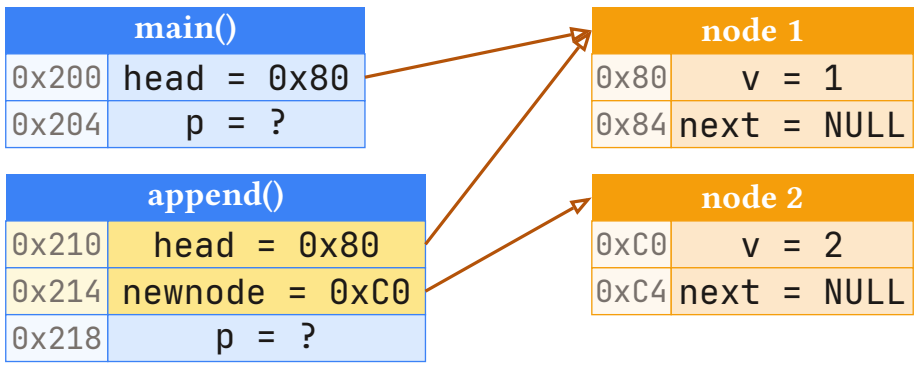
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode->next = NULL;
14    if (p == NULL)
15      return newnode;
16    p->next = newnode;
17    return head;
18  }
```

[1, ]



# append(0x80, 0xC0)

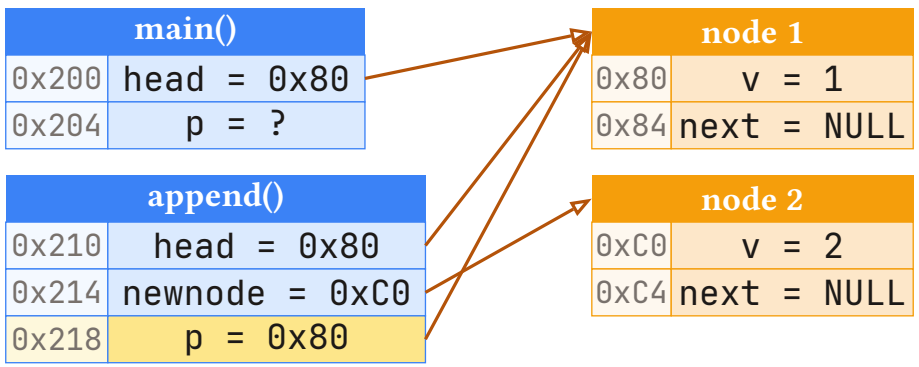
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```



# p = 0x80 (last node)

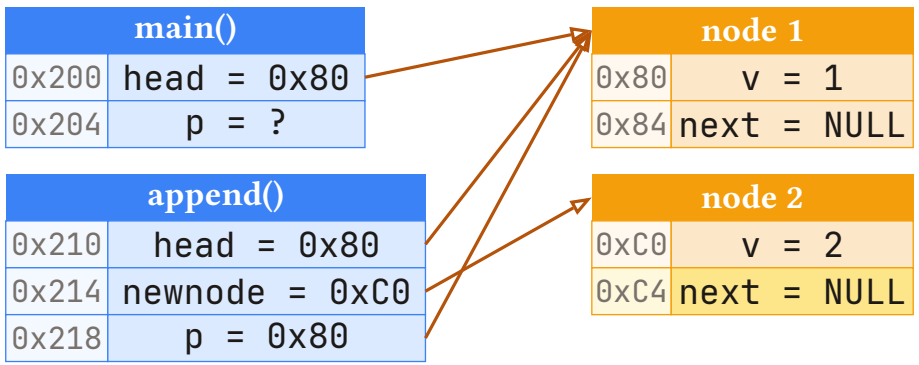
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode->next = NULL;
14    if (p == NULL)
15      return newnode;
16    p->next = newnode;
17    return head;
18  }
```

find_last(0x80) ⟶ 0x80



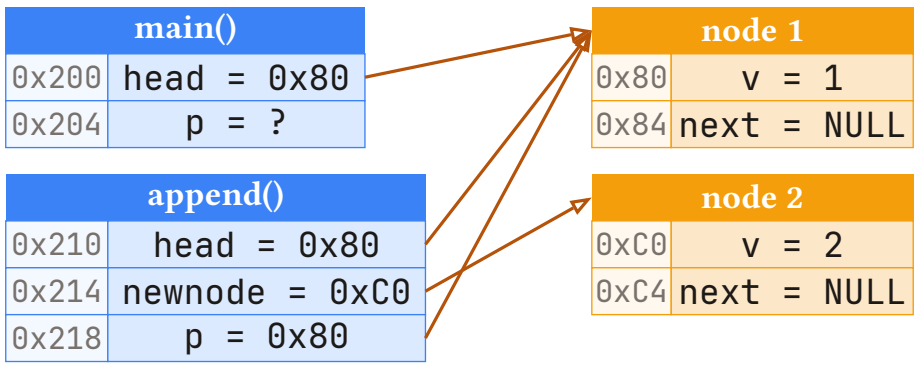
# newnode→next = NULL

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```



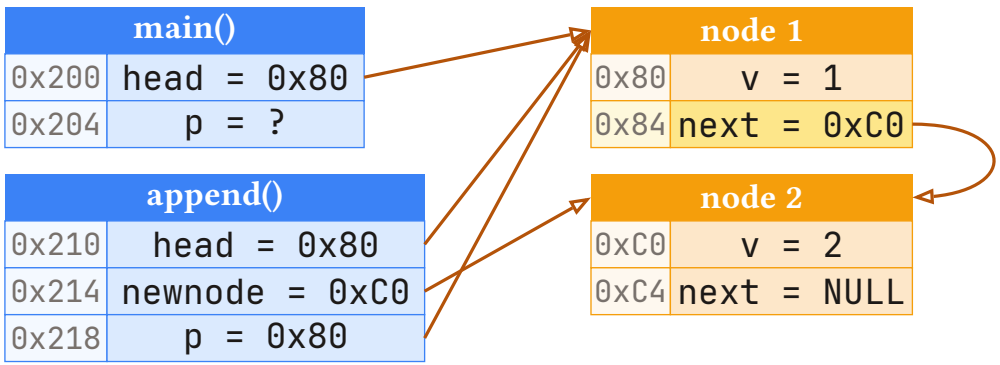
# p ≠ NULL

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```



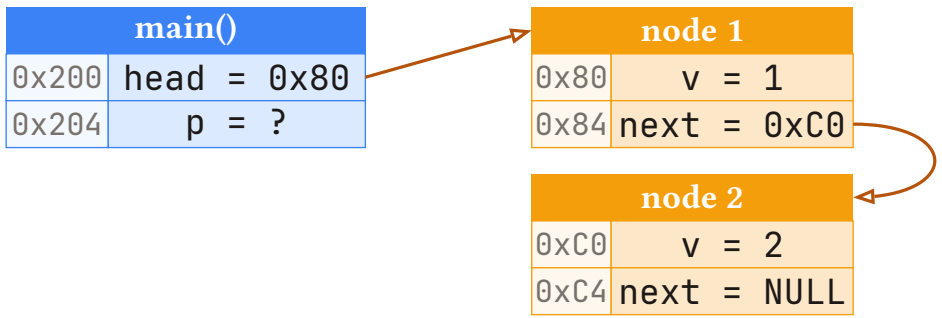
# p→next = newnode

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```



# return head

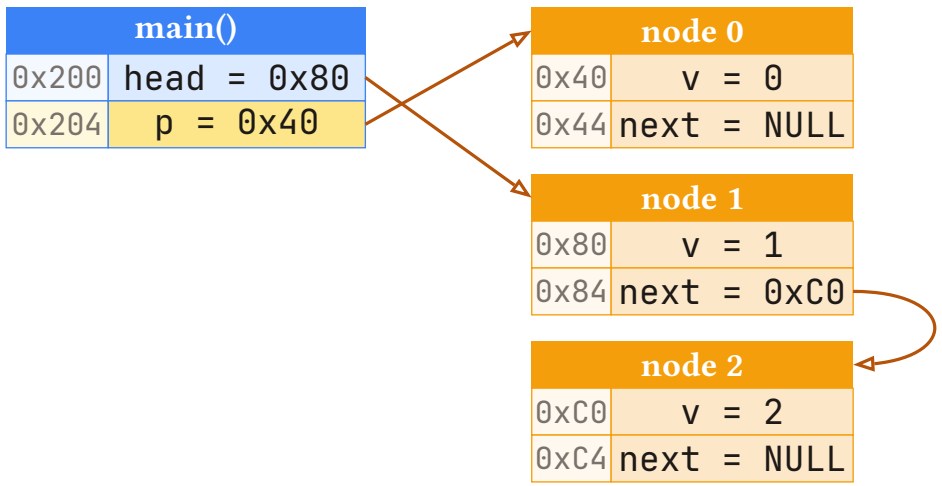
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *append(node *head, node *newnode) {
12    node *p = find_last(head);
13    newnode→next = NULL;
14    if (p == NULL)
15      return newnode;
16    p→next = newnode;
17    return head;
18  }
```




`head` unchanged. `[1, 2, ]`

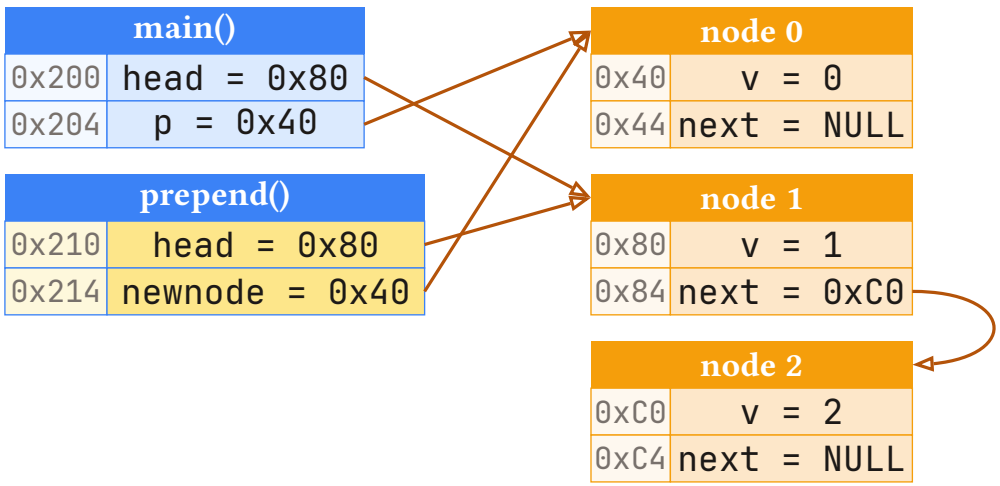
# p0 on [1, 2]: new node

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *prepend(node *head, node *newnode) {
12    newnode→next = head;
13    return newnode;
14  }
```



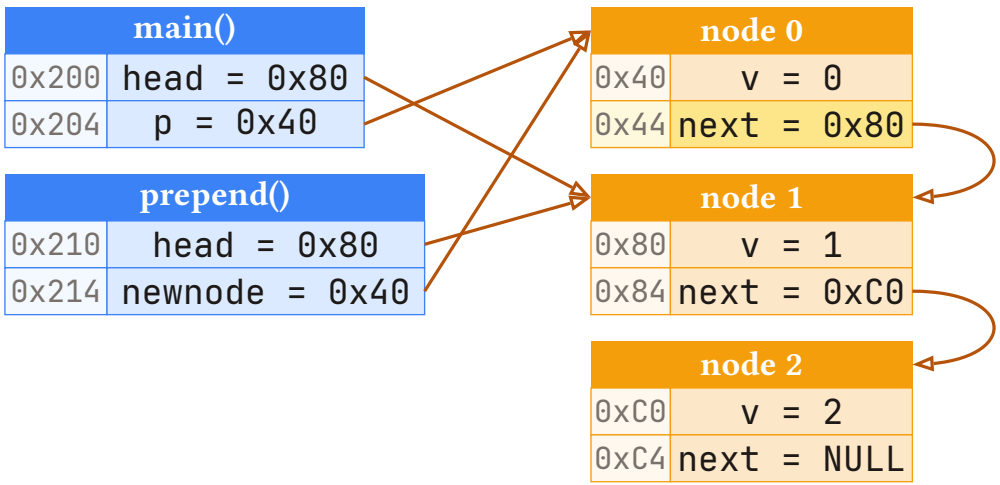
# prepend gets copies

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *prepend(node *head, node *newnode) {
12    newnode→next = head;
13    return newnode;
14  }
```



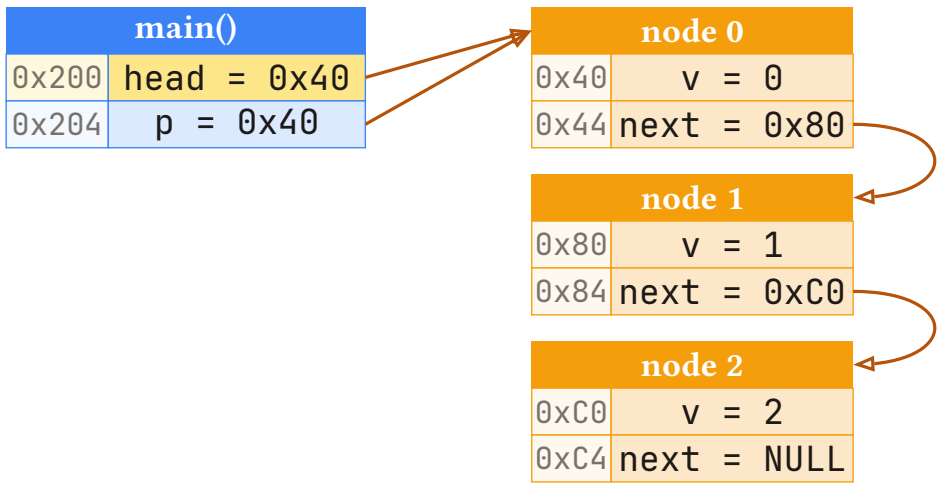
# newnode→next = head

```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *prepend(node *head, node *newnode) {
12    newnode→next = head;
13    return newnode;
14  }
```



# `return newnode` ⟶ main's `head`

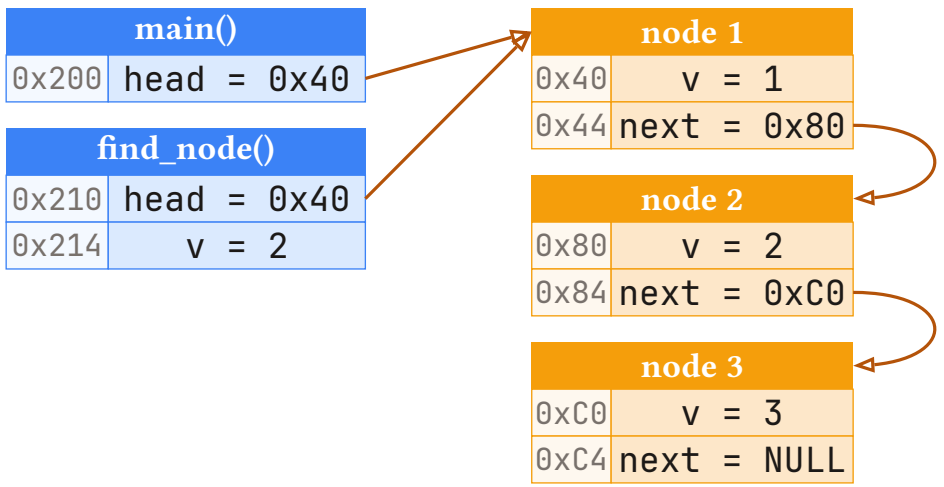
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5
6     node *p = new_node(0);
7     head = prepend(head, p);
8     return 0;
9   }
10
11  node *prepend(node *head, node *newnode) {
12    newnode→next = head;
13    return newnode;
14  }
```




Without `head =` in `main`: `head` stays `0x80`, node 0 lost.

# 2 on [1, 2, 3]: find_node

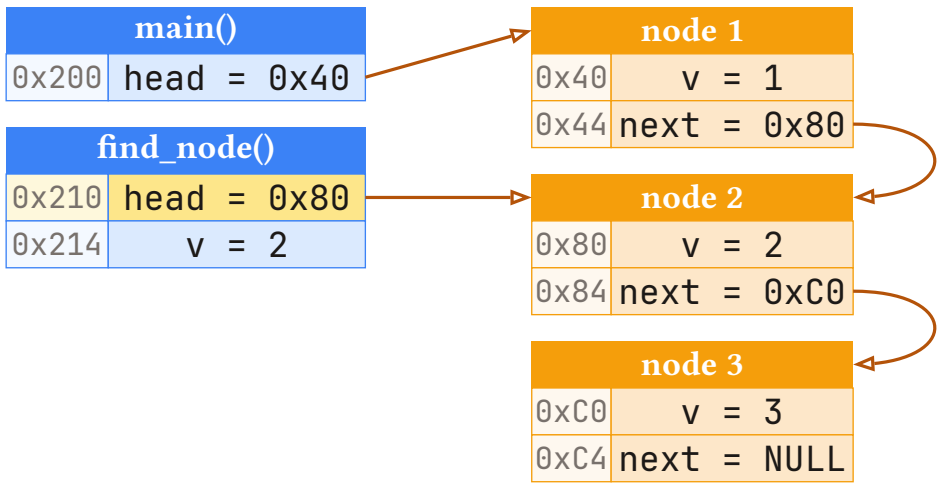
```
1   node *find_node(node *head, int v) {
2     while (head ≠ NULL) {
3       if (head→v == v)
4         return head;
5       head = head→next;
6     }
7     return head;
8   }
```




1 ≠ 2

# head = head→next

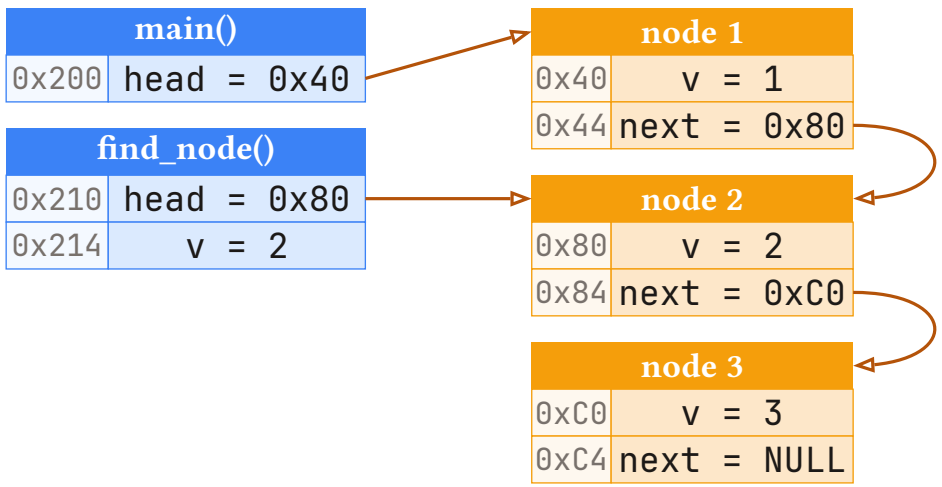
```
1   node *find_node(node *head, int v) {
2     while (head != NULL) {
3       if (head→v == v)
4         return head;
5       head = head→next;
6     }
7     return head;
8   }
```




Local `head` moves. `main`'s `head` stays.

# Match: `return 0x80`

```
1   node *find_node(node *head, int v) {
2     while (head != NULL) {
3       if (head->v == v)
4         return head;
5       head = head->next;
6     }
7     return head;
8   }
```

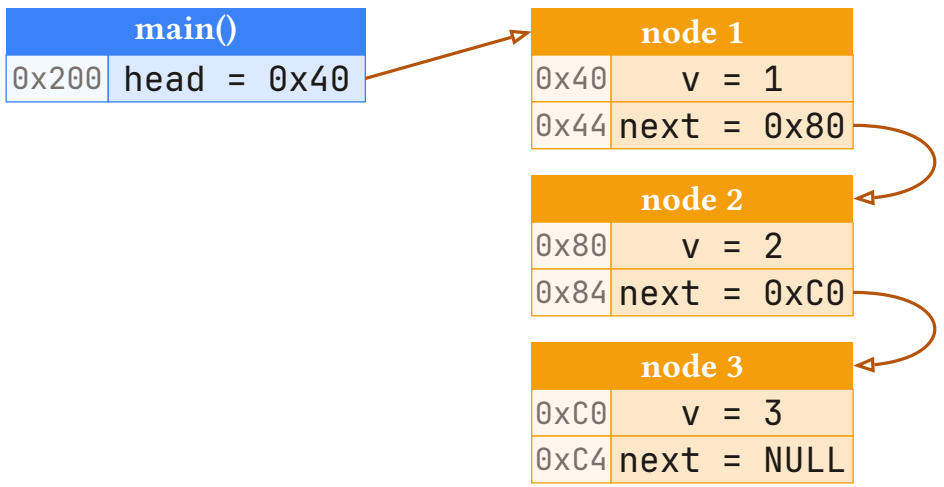



```
0x80: v=2     next=0xC0
```

```
/* Part 1: delete_node */
```

# d2: before

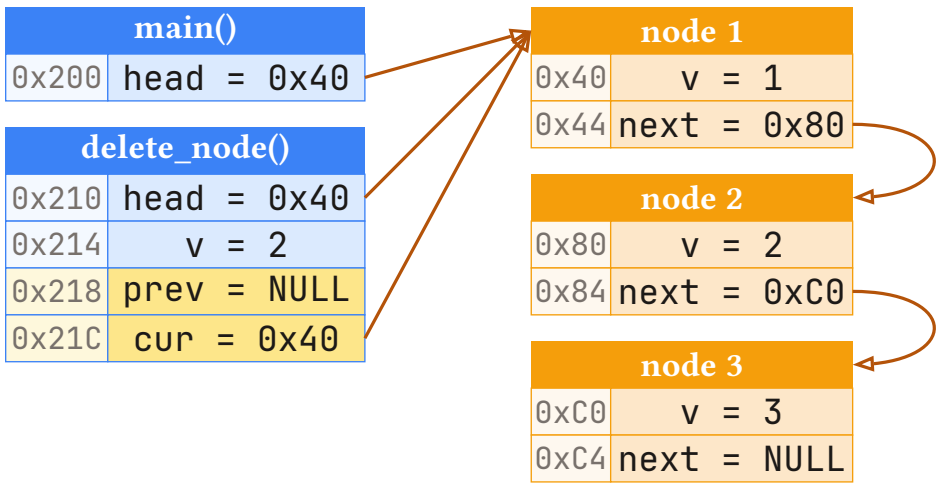
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5     head = append(head, new_node(3));
6
7     head = delete_node(head, 2);
8     return 0;
9   }
```




[1, 2, 3, ] ⟶ [1, 3, ]

I'll only show what your code should do. Writing the actual code is your job.

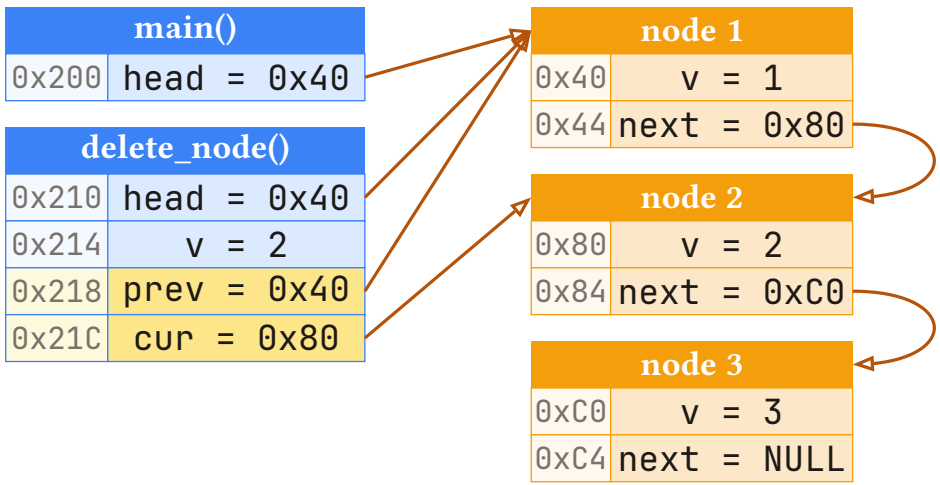
# Walk with two pointers




$1 \neq 2$

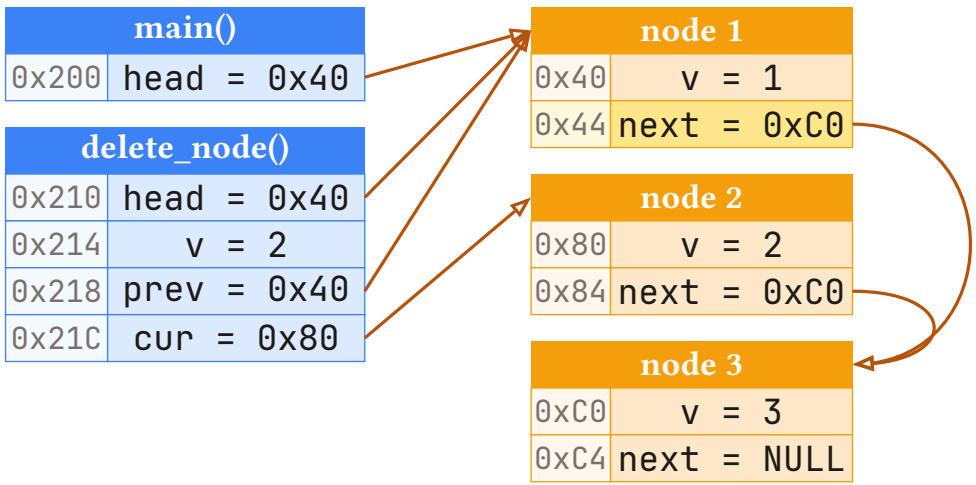
I'll only show what your code should do. Writing the actual code is your job.

## `prev = cur; cur = cur→next`



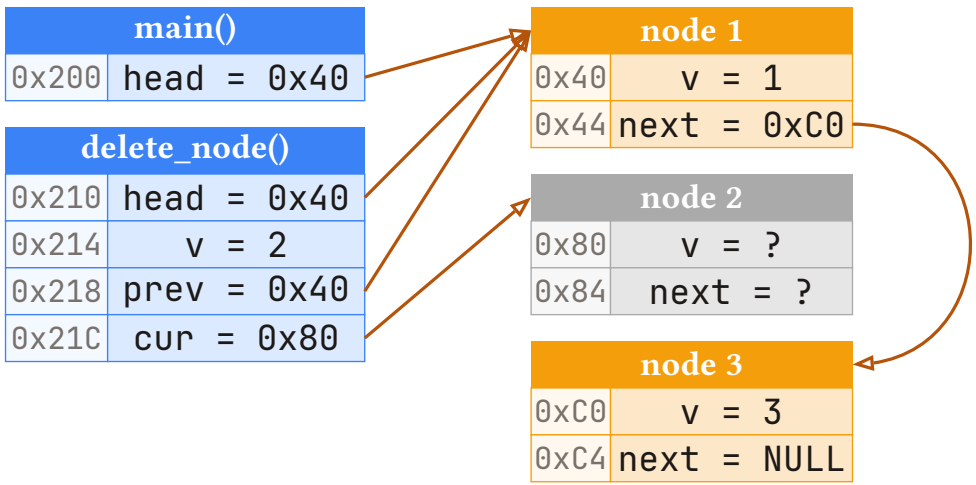

Match. `prev` = node **before** it.

## `prev→next = cur→next`




Node 2 unreachable, still allocated

# free(cur)




```
[1, 3, ]
```

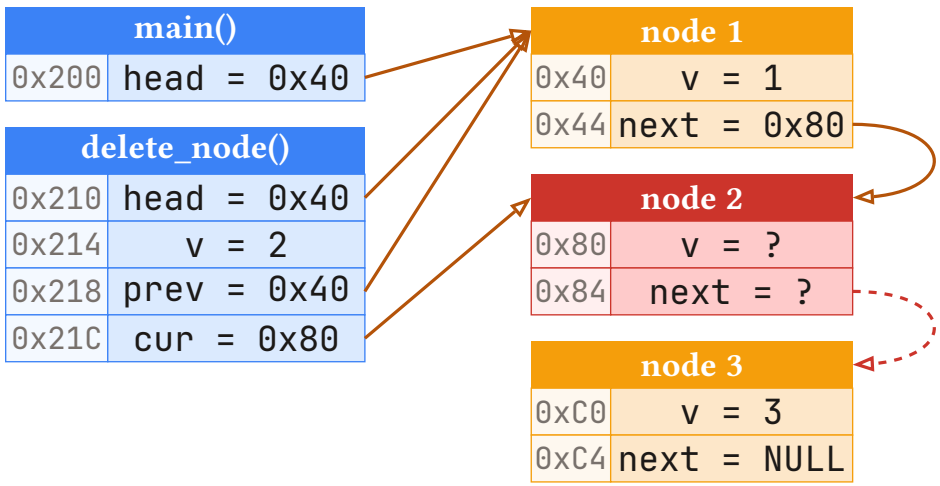
I'll only show what your code should do. Writing the actual code is your job.

# What's wrong here?

```
free(cur);
prev→next = cur→next;
```

I'll only show what your code should do. Writing the actual code is your job.

# Answer: reads a freed node




`cur→next` after `free(cur)` ⟶ use-after-free. Rewire first, then `free`.

# d4: not on the list

```
d4
linkedlist: The number is not on the list.
[1, 2, 3, ]
```

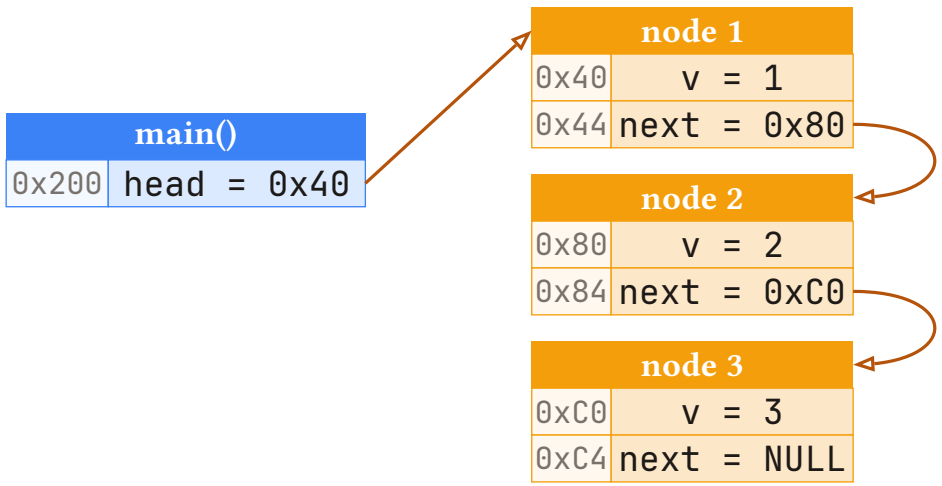
```
error_message(ERR_NOTFOUND);
```

Same for empty list

```
/* Part 2: reverse_list */
```

# r: before

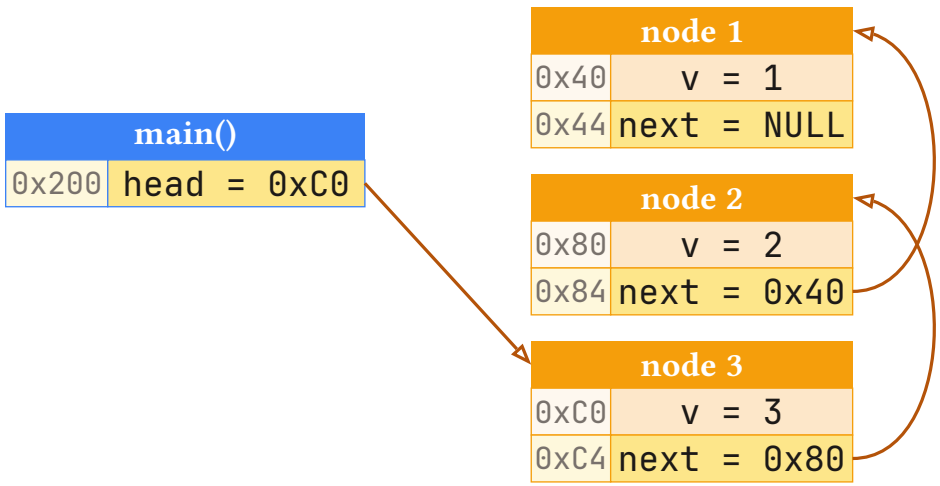
```
1   int main(void) {
2     node *head = NULL;
3     head = append(head, new_node(1));
4     head = append(head, new_node(2));
5     head = append(head, new_node(3));
6
7     head = reverse_list(head);
8     return 0;
9   }
```




[1, 2, 3, ] ⟶ [3, 2, 1, ]

I'll only show what your code should do. Writing the actual code is your job.

# r: after




`[3, 2, 1, ]` – same nodes, same addresses. No `malloc`, no changing `v`.

# Check with `i`

```
i
head = 0x40
0x40: v=1     next=0x80
0x80: v=2     next=0xC0
0xC0: v=3     next=(nil)
[1, 2, 3, ]
```

```
r
[3, 2, 1, ]
i
head = 0xC0
0xC0: v=3     next=0x80
0x80: v=2     next=0x40
0x40: v=1     next=(nil)
[3, 2, 1, ]
```

Segfault? `-fsanitize=address` in Makefile `CFLAGS`

I'll only show what your code should do. Writing the actual code is your job.

# Questions?

**Office hours (Hudson)**

Monday, 12:15 PM to 2:15 PM @ ITE 214

**Discord**

Ask any time

I will try to be quick!

Credits:

Slides made with Typst (typst.app) w/ help from Gen. AI